



EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN

WEEK 3

Modelling Language — UML

PROGRAMME	BSc Information Communication and Technology
COURSE CODE / YEAR	BICT 3202 · Year 3
LECTURER	Francis Fweta — ICT Lecturer
DELIVERY	2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning

Prepared by Francis Fweta · Exploits University · Week 3 of 16



Contents

1. Week 3 Overview	3
2. Learning Outcomes	3
3. What Is UML?	3
4. What UML Is Not	4
5. Why UML Is Important in OOAD	4
6. The Building Blocks of UML	5
7. Building Block 1 — Things	5
8. Building Block 2 — Relationships	7
9. Building Block 3 — Diagrams	9
10. Rules of UML	13
11. UML Common Mechanisms	13
12. UML Architecture — the Four Layers	14
13. Worked Example — University Registration	15
14. Independent Learning (10 Hours)	15
15. Week 3 Summary	16
16. Examination-Style Questions	16
17. References and Further Reading	17

1. Week 3 Overview

Week 1 moved us from business problems to business needs and software requirements. Week 2 taught us to think about the problem in terms of objects, classes, attributes, behaviour, relationships, abstraction, encapsulation, modularity and complexity management.

Now we face a practical problem: **how do we communicate all these models in a standard way** so that analysts, developers, clients and other stakeholders can understand them? The answer is **UML — the Unified Modeling Language**.

The course outline places five topics here: an overview of UML, the building blocks of UML, the rules of UML, the mechanisms of UML, and UML architecture. These form the conceptual foundation needed before drawing formal UML diagrams.

The Object Management Group (OMG) describes UML as a standard language for visualising, specifying and documenting software and systems, including their structures, behaviours and processes. The current formal specification listed by OMG is **UML 2.5.1**, adopted in December 2017.

2. Learning Outcomes

By the end of this week, students should be able to:

- Define UML and explain why it is used in OOAD.
- Explain the difference between a modelling language and a development methodology.
- Explain what UML can and cannot do.
- Identify the major building blocks — things, relationships and diagrams.
- Explain structural, behavioural, grouping and annotational elements.
- Identify major UML diagram categories and explain their purposes.
- Explain UML rules, well-formedness, syntax and semantics.
- Explain the common mechanisms — specifications, adornments, common divisions and extensibility.
- Explain stereotypes, tagged values and constraints.
- Explain the four-layer UML architecture (M0–M3) and why it matters.
- Apply UML concepts to a university registration system.

3. What Is UML?

UML stands for **Unified Modeling Language**. Break the name into three parts: **Unified** — it brought together ideas from several earlier object-oriented modelling approaches; **Modeling** — it is used to create representations of systems; **Language** — it provides defined concepts and notation for communicating those representations.

DEFINITION

UML is a standardised modelling language used to represent, communicate and document systems.

What problem does UML solve? Imagine five people developing a university system — a business analyst, a software architect, a programmer, a database designer and an administrator. The analyst says “The Student is associated with a Course.” The programmer asks what “associated” means; the administrator thought a student could register for many courses; the database designer asks whether it is one-to-many or many-to-many. A UML diagram makes the intended structure **visible**, giving the team something concrete to discuss.

4. What UML Is Not

4.1 Not a Programming Language

Students sometimes hear “language” and assume UML is like Java, Python, C++ or JavaScript. It is not. A programming language tells a computer how to execute instructions; UML tells humans and modelling tools how to represent and communicate the structure and behaviour of a system.

4.2 Not a Development Methodology

A modelling language answers “**How do I represent the system?**”. A development methodology answers “**What activities should we perform, in what order, and how should the system be developed?**” UML itself does not prescribe one complete software development process; processes such as the Unified Process (UP) can use UML.

KEY DISTINCTION

UML = modelling language. UP / USDP = development process (studied in Weeks 12–13).

4.3 Not Only for Software

Although UML is heavily associated with software engineering, it can also model business processes, organisational structures, workflows and non-software systems. A hospital workflow can be modelled before deciding exactly what software will implement it.

5. Why UML Is Important in OOAD

- **Understand requirements** — use cases represent what users expect.
- **Model structure** — class diagrams show classes and relationships.
- **Model behaviour** — state and activity diagrams represent change and workflows.
- **Model interactions** — sequence and communication diagrams show how objects communicate.
- **Communicate designs** — all stakeholders discuss the same model.
- **Document systems** — models become technical documentation.

- **Support maintenance** — future developers read the models to understand the system.

ONE DIAGRAM CANNOT EXPLAIN EVERYTHING

A single diagram cannot show all classes, users, workflows, interactions, databases, servers, states and requirements at once. UML therefore provides **different diagram types for different perspectives** — a principle known as using multiple views/models.

6. The Building Blocks of UML

The classic conceptual way of learning UML (the framework of Booch, Rumbaugh and Jacobson) is to think in terms of three building blocks: **Things**, **Relationships** and **Diagrams**.

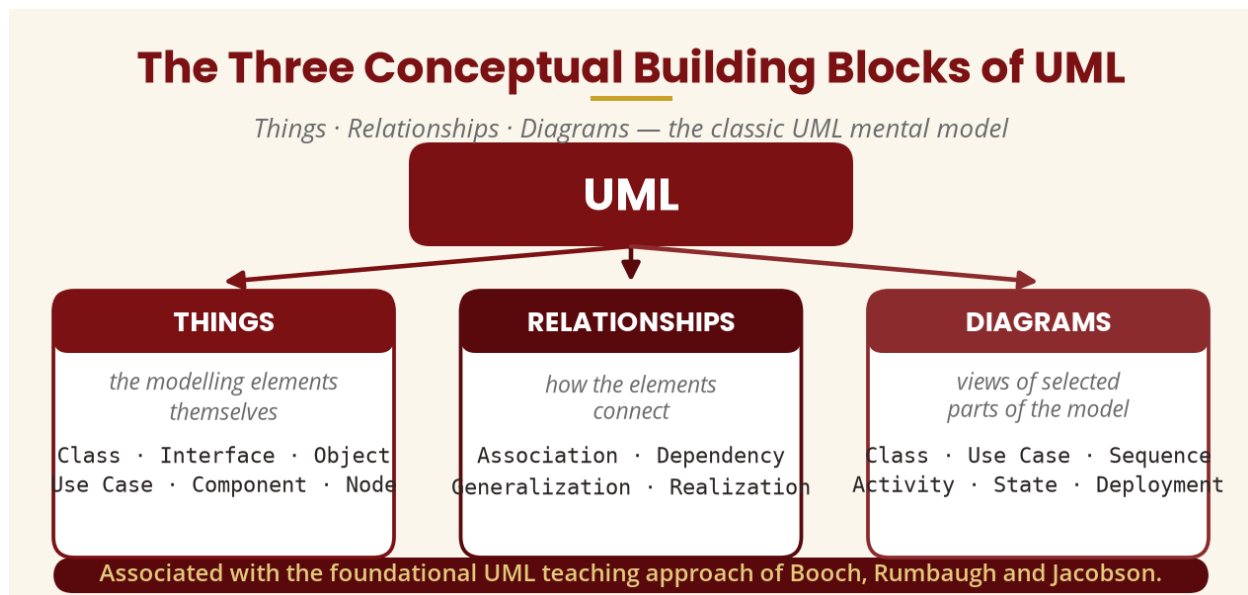


Figure 1 — The three conceptual building blocks of UML

7. Building Block 1 — Things

A **thing** is an important modelling element or abstraction. For teaching purposes, things are grouped into **structural**, **behavioural**, **grouping** and **annotational**.

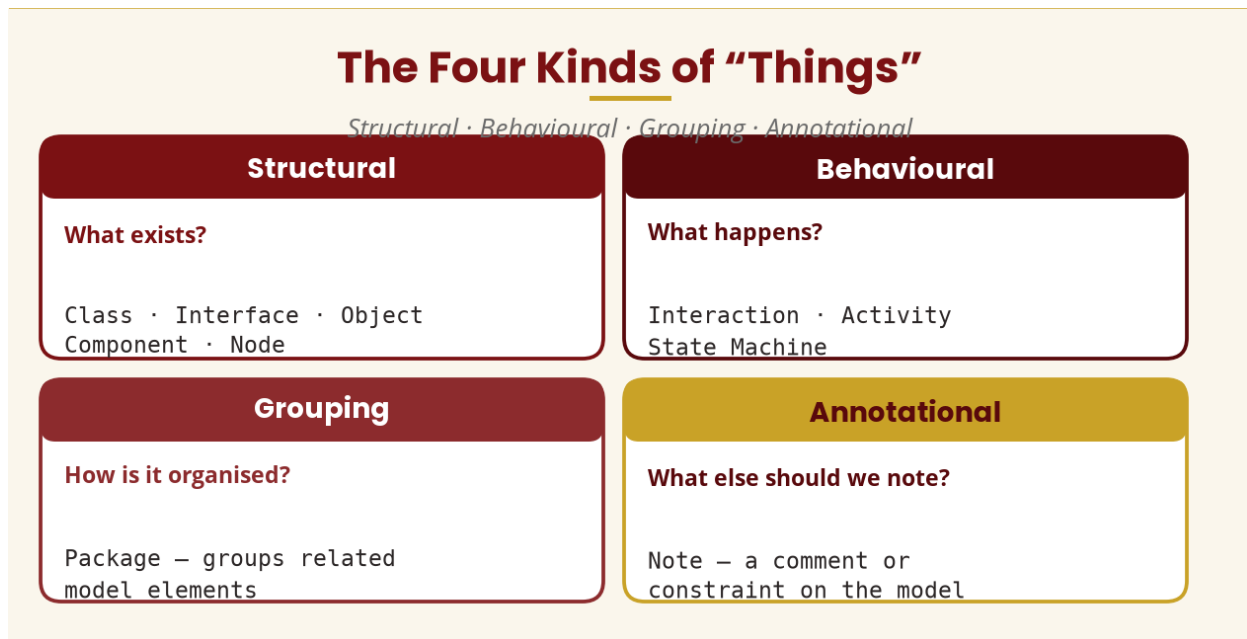


Figure 2 — The four kinds of UML things

7.1 Structural Things

Structural things answer “**What exists?**” and include classes, interfaces, objects, components and nodes.

A **class** represents a set/type of objects sharing common characteristics and behaviour — shown as a three-compartment box: name, attributes and operations.

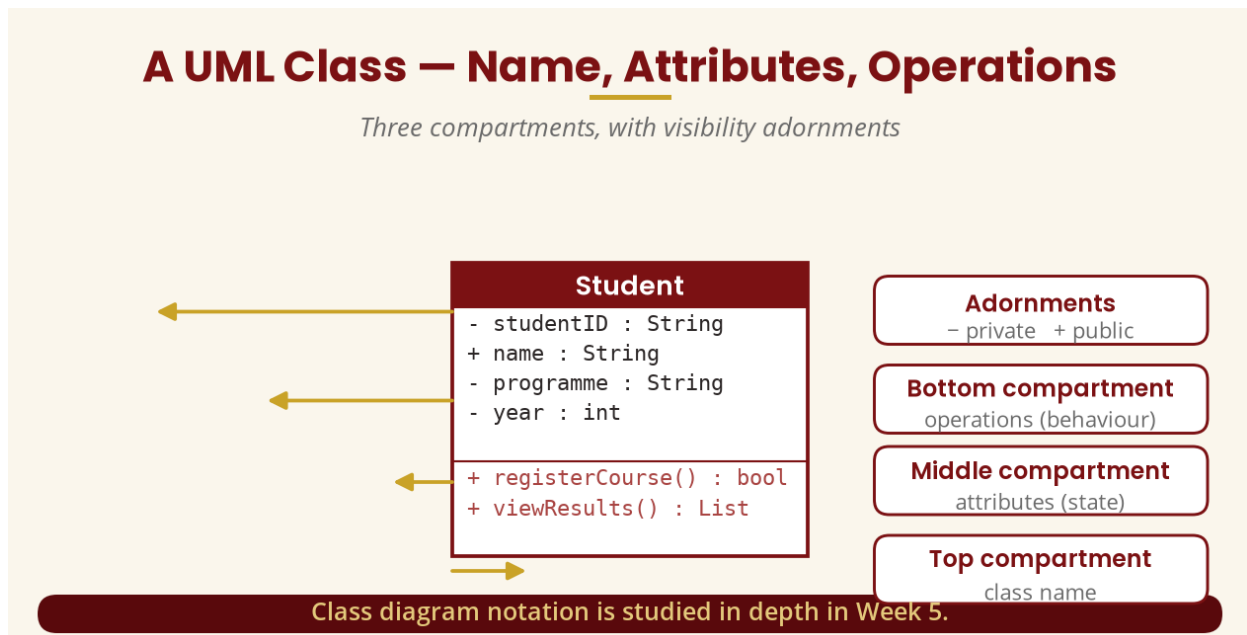


Figure 3 — A UML class with its three compartments and visibility adornments

An **interface** specifies a set of operations that a class or component can provide; different providers can implement it differently (BankPayment, MobileMoneyPayment, CardPayment all realising PaymentService). An **object** is a particular instance (student001 : Student). A **component** is a modular, replaceable part of a system. A **node** is a computational or physical resource — a server, device or execution environment — used in deployment diagrams.

Interface & Realization

An interface is a contract — different classes can fulfil it



The user of the interface does not need to know the internal implementation.

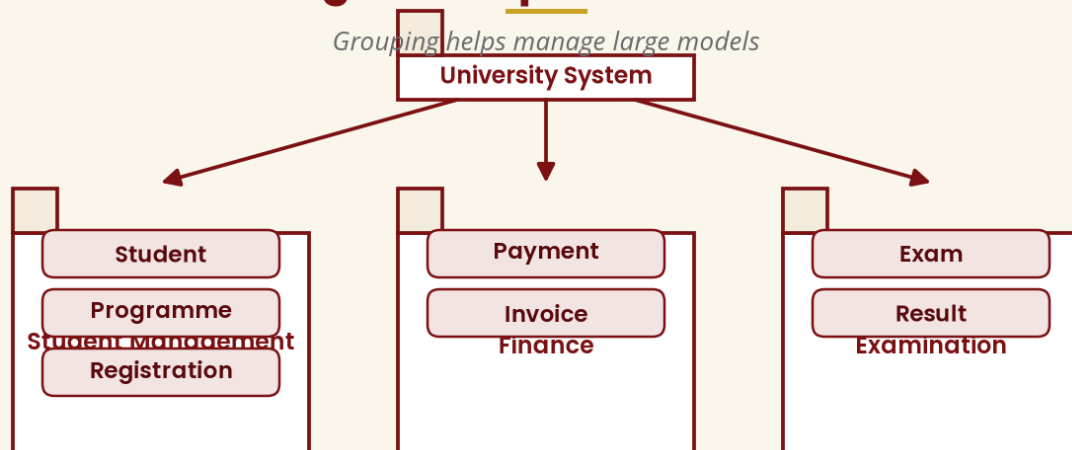
Figure 4 — An interface and three alternative realizations

7.2 Behavioural, Grouping and Annotational Things

Behavioural things answer “What happens?” — interactions, activities and state machines. **Grouping things** organise other elements; the most important is the **package**, which groups related elements (Student Management, Finance, Examination packages). **Annotational things** add extra information — the simplest is a **note**, used to explain a business rule, constraint or design decision.

Packages Group Model Elements

Grouping helps manage large models



Packages directly connect to Week 2's discussion of complexity management.

Figure 5 — Packages group model elements and manage large models

8. Building Block 2 — Relationships

Relationships connect modelling elements. The four important UML relationships are **association**, **dependency**, **generalization** and **realization** (with include/extend in use cases and aggregation/composition in structural modelling coming later).

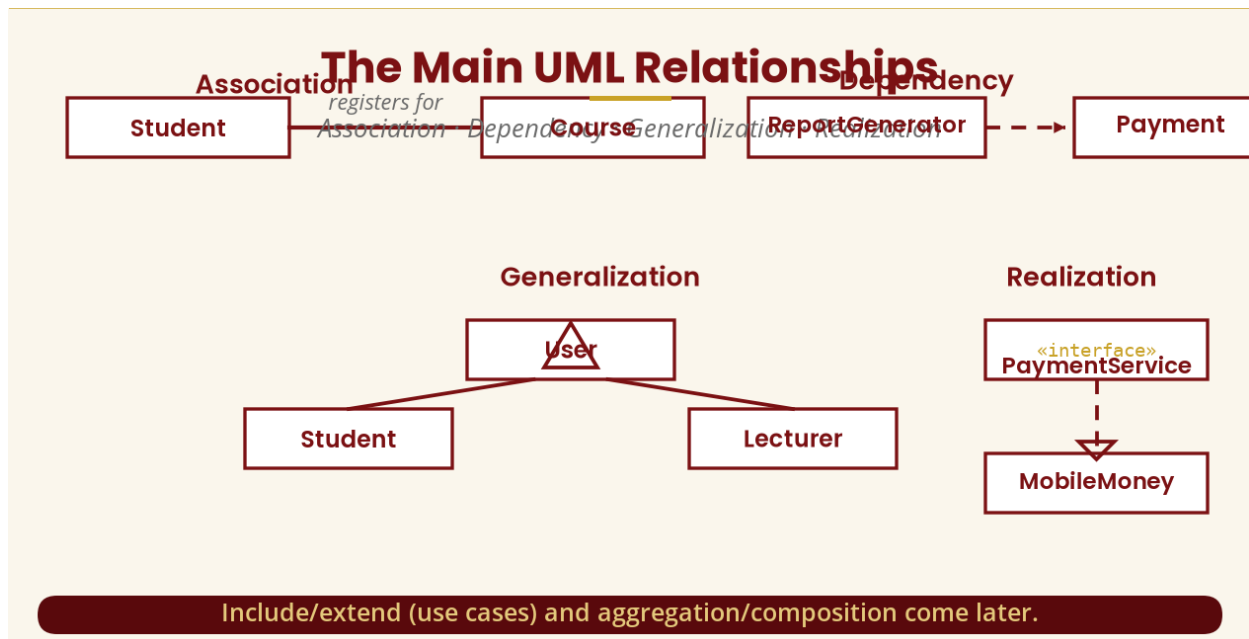


Figure 6 — The four main UML relationship types

Association is a structural relationship between classifiers (Student — registers for — Course). **Dependency** (dashed open arrow) means one element depends on another. **Generalization** (hollow triangle to the general classifier) is a general-to-specific relationship — Student and Lecturer are specialised types of User. **Realization** (dashed hollow triangle) means a classifier fulfils a contract, commonly an interface.

8.1 Multiplicity

An association can specify how many instances participate. **Student 1 ——— 0..* Course** means one Student can be associated with zero or many Courses.

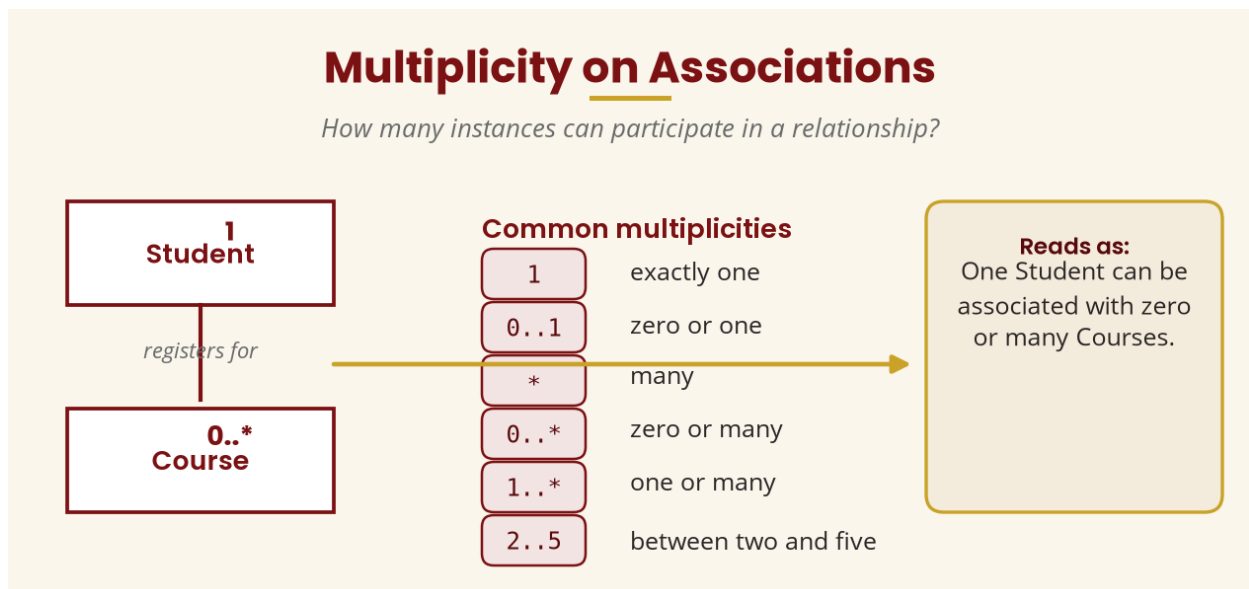


Figure 7 — Multiplicity notation and its meaning

9. Building Block 3 — Diagrams

A **diagram** provides a visual representation of selected parts of a model. The important point: **a diagram is a view of a model, not necessarily the entire model** — which is why large systems require multiple diagrams.

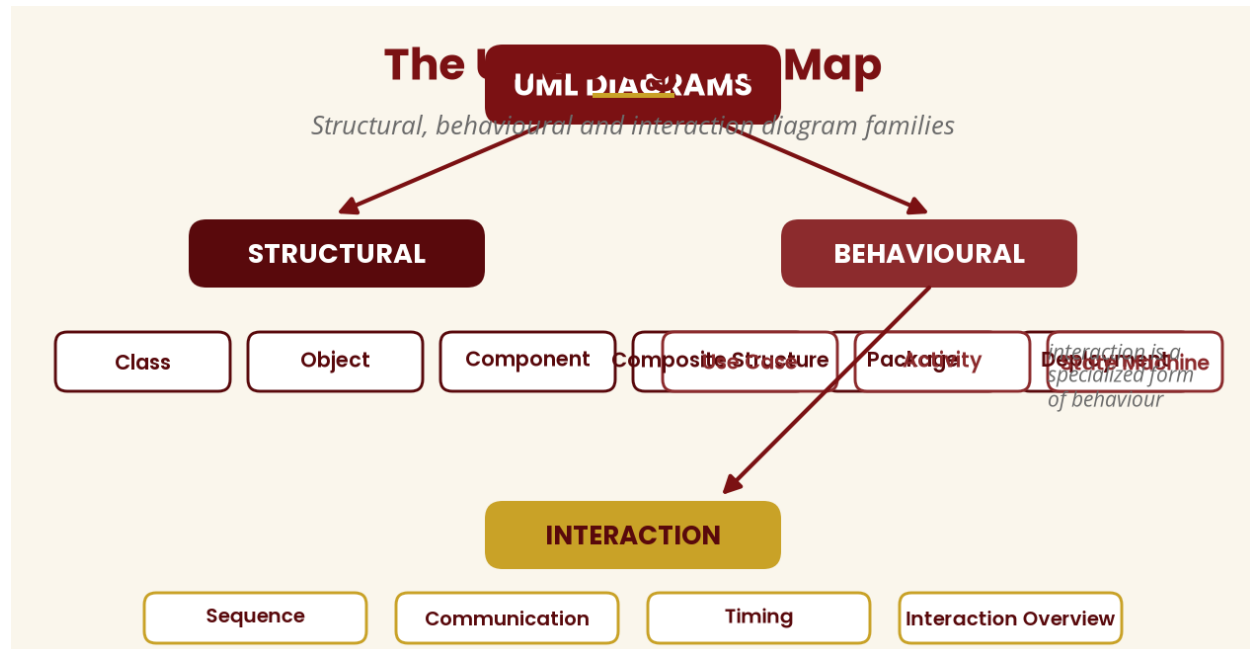


Figure 8 — The UML diagram families

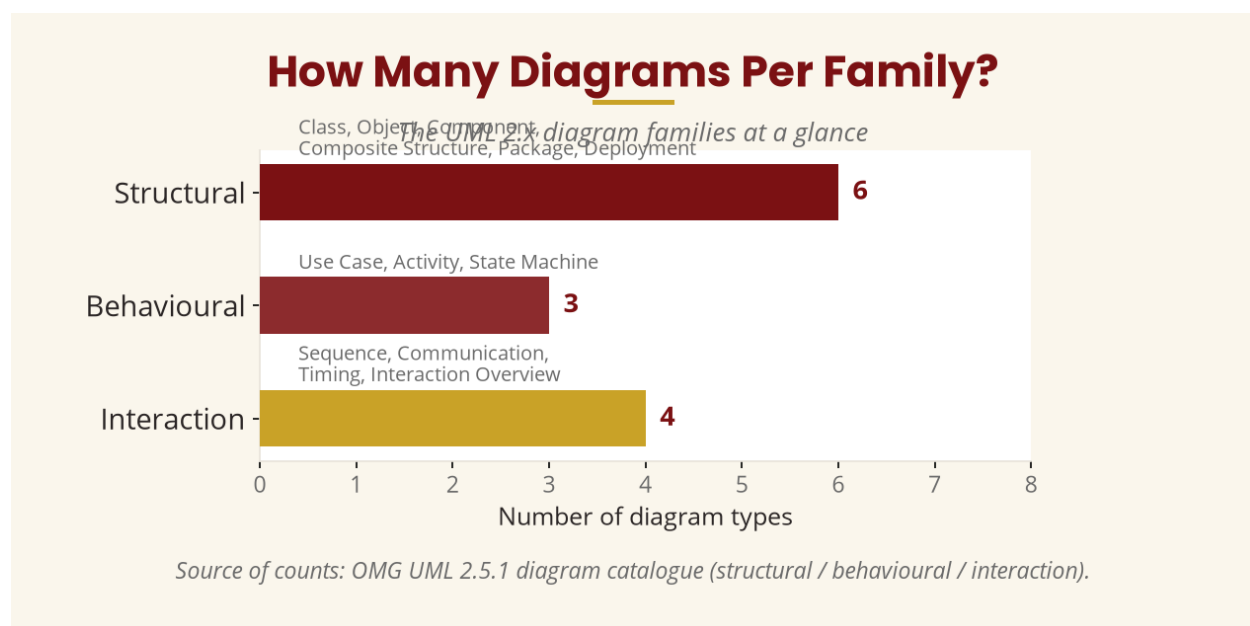


Figure 9 — Diagram types per family (per OMG UML 2.5.1)

Diagram	Main question
Use Case	What does the user want to accomplish?
Class	What structural elements exist?
Object	What are particular instances?

Diagram	Main question
Sequence	How do participants communicate over time?
Communication	Which objects communicate and how?
Activity	What workflow occurs?
State Machine	How does an entity change state?
Component	What major software components exist?
Deployment	Where does the software run?
Package	How are model elements organised?

9.1 Example Diagrams

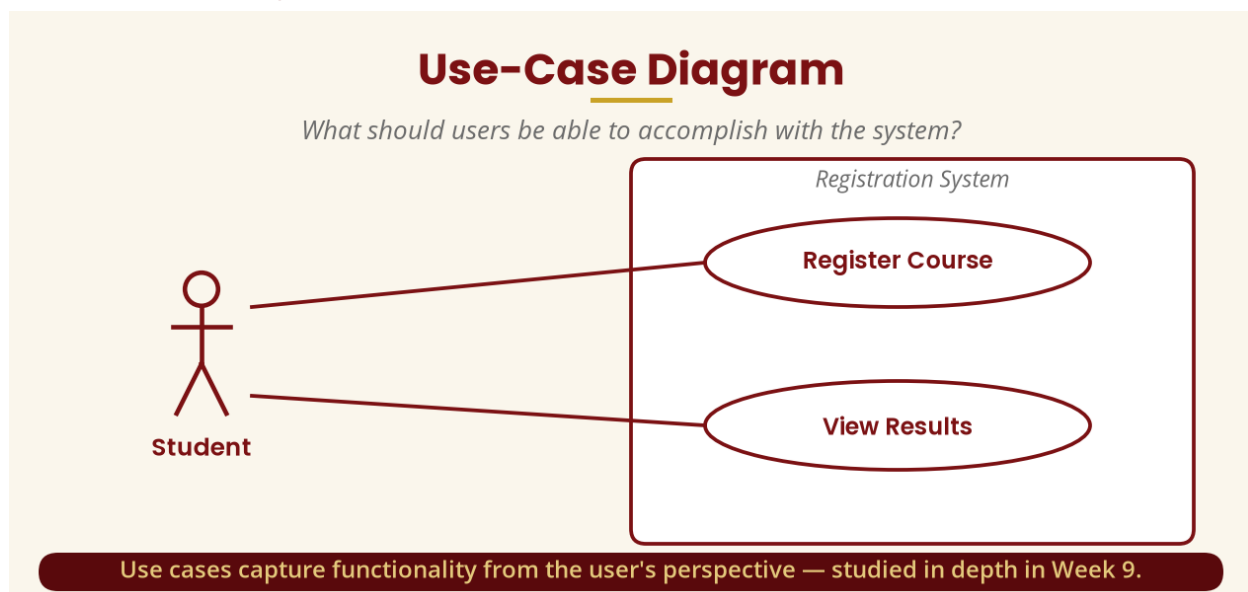


Figure 10 — Use-case diagram: what users can accomplish

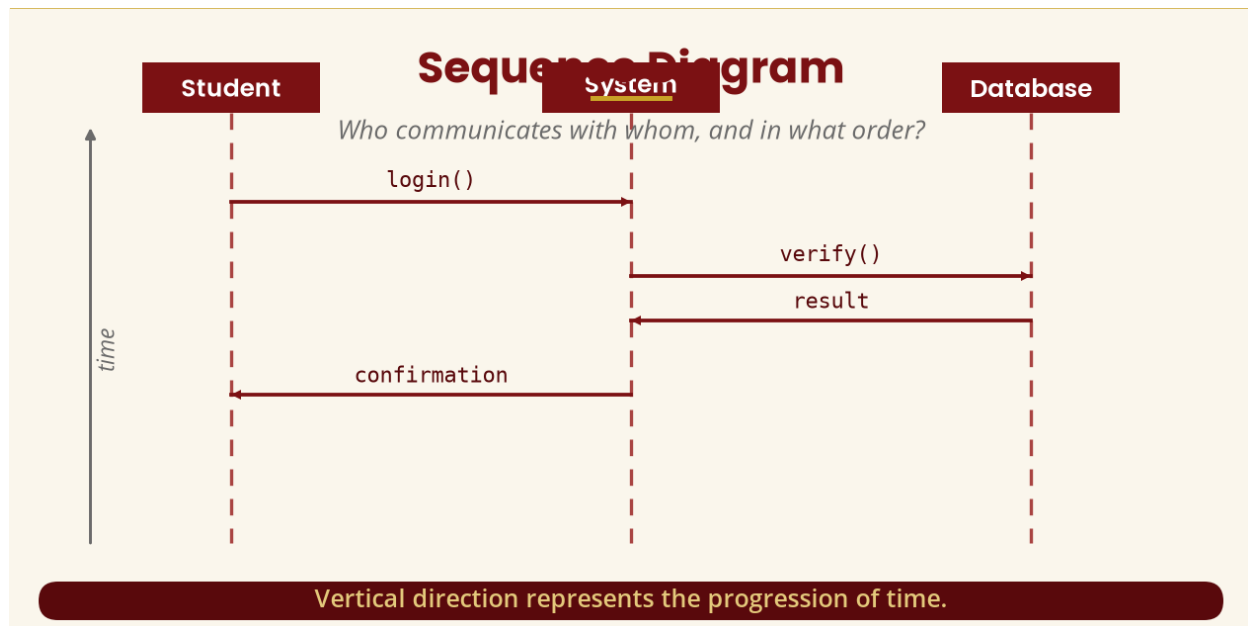


Figure 11 — Sequence diagram: who communicates, in what order

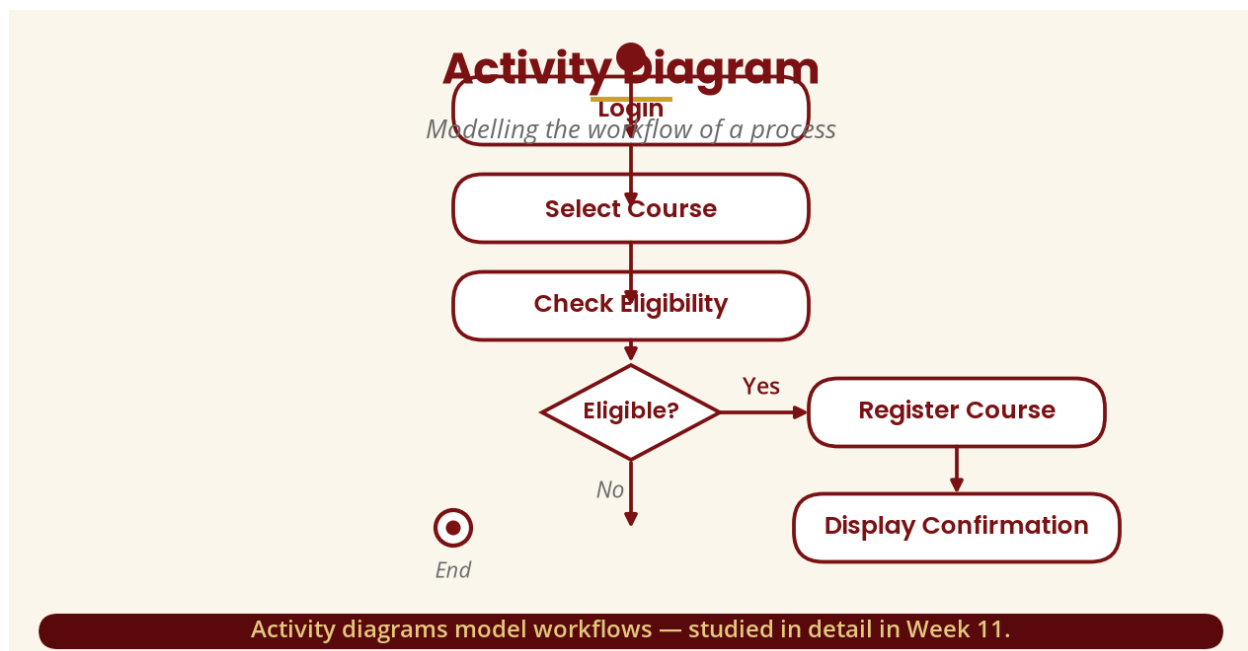


Figure 12 — Activity diagram: the workflow of a process

State Machine Diagram

How an object changes state as events occur



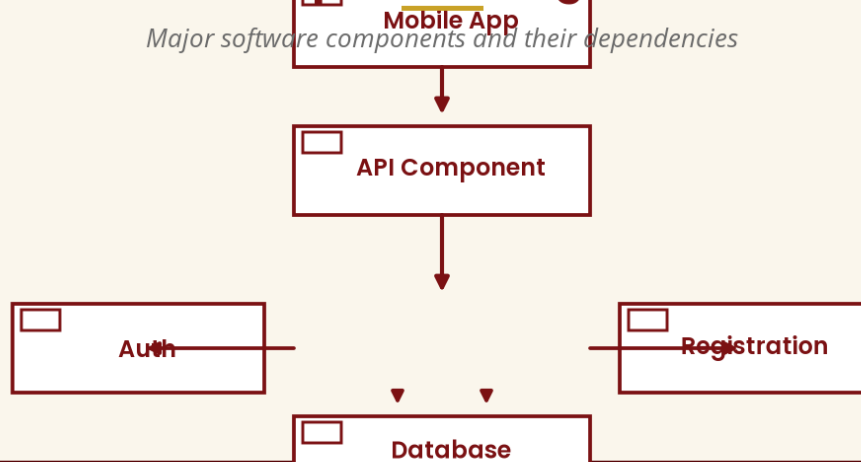
An event causes each transition.

State modelling is developed fully in Weeks 7 and 8.

Figure 13 — State machine: how an object changes state

Component Diagram

Major software components and their dependencies

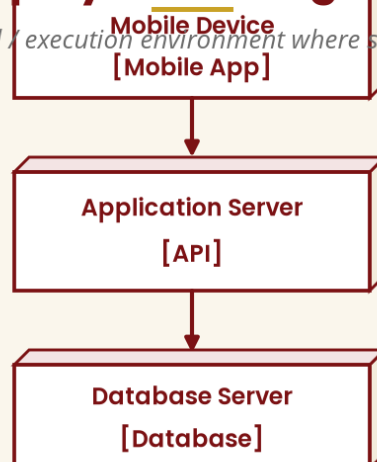


Components relate to design and architecture more than early requirements analysis.

Figure 14 — Component diagram: major software components

Deployment Diagram

The physical / execution environment where software runs



Node = server,
device, execution
environment or
hardware resource

**3-tier
architecture**
client, server, database

Figure 15 — Deployment diagram: where software runs

10. Rules of UML

UML is not simply a collection of shapes — there are rules governing what elements mean, how they relate, how notation is used, and what constitutes a well-formed model.

Syntax concerns how something is represented (*how is it drawn?*); **semantics** concerns what it means (*what does the representation mean?*). A diagram can look attractive yet communicate the wrong meaning.

WELL-FORMEDNESS

A model is **well-formed** when it satisfies the relevant rules of the modelling language. You cannot arbitrarily connect unrelated elements and claim the result has the intended UML meaning.

FOUR QUESTIONS WHEN CHECKING A MODEL

1. Is the notation correct? 2. Does the relationship make semantic sense? 3. Does the model satisfy the requirements? 4. Can another person understand it? — All four matter.

11. UML Common Mechanisms

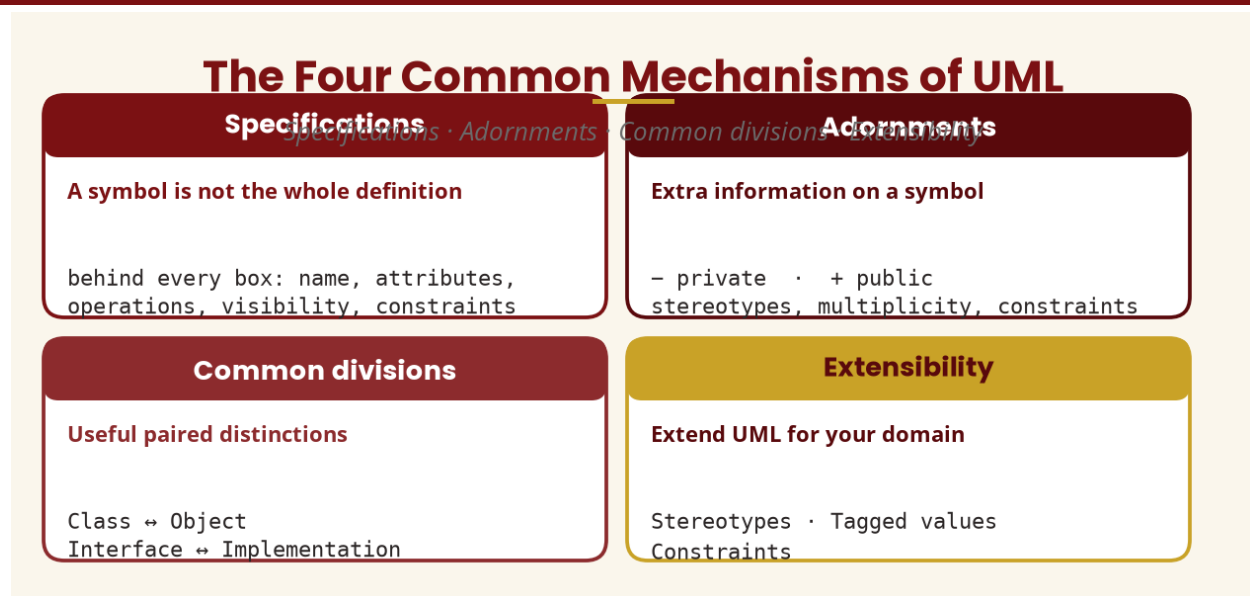


Figure 16 — The four common mechanisms

Specifications: a graphical symbol is not the complete definition — behind the Student box lies a specification of name, attributes, operations, visibility, relationships and constraints.

Adornments: extra information on a symbol (the – and + visibility markers, stereotypes, multiplicity, constraints). **Common divisions:** paired distinctions such as class↔object and interface↔implementation. **Extensibility:** mechanisms for customising UML for particular domains.

11.1 Extensibility — Stereotypes, Tagged Values, Constraints

A **stereotype** gives an element a more specialised meaning — «entity» Student, «controller» RegistrationController, «boundary» RegistrationPage. A **tagged value** provides additional property information — {author = "Analysis Team"}, {location = "Data Centre"}. A **constraint** is a condition that must be satisfied — {age ≥ 18}, {balance ≥ 0}, or {student must satisfy prerequisite}. Constraints make rules explicit: Student—registers—Course alone does not say that the prerequisite must have been passed; {PrerequisiteSatisfied = true} does.

12. UML Architecture — the Four Layers

“UML architecture” here means the architectural organisation of the UML language itself — its layered meta-modelling structure — **not** three-tier or client-server software architecture. The four commonly discussed levels are M3 □ M2 □ M1 □ M0.

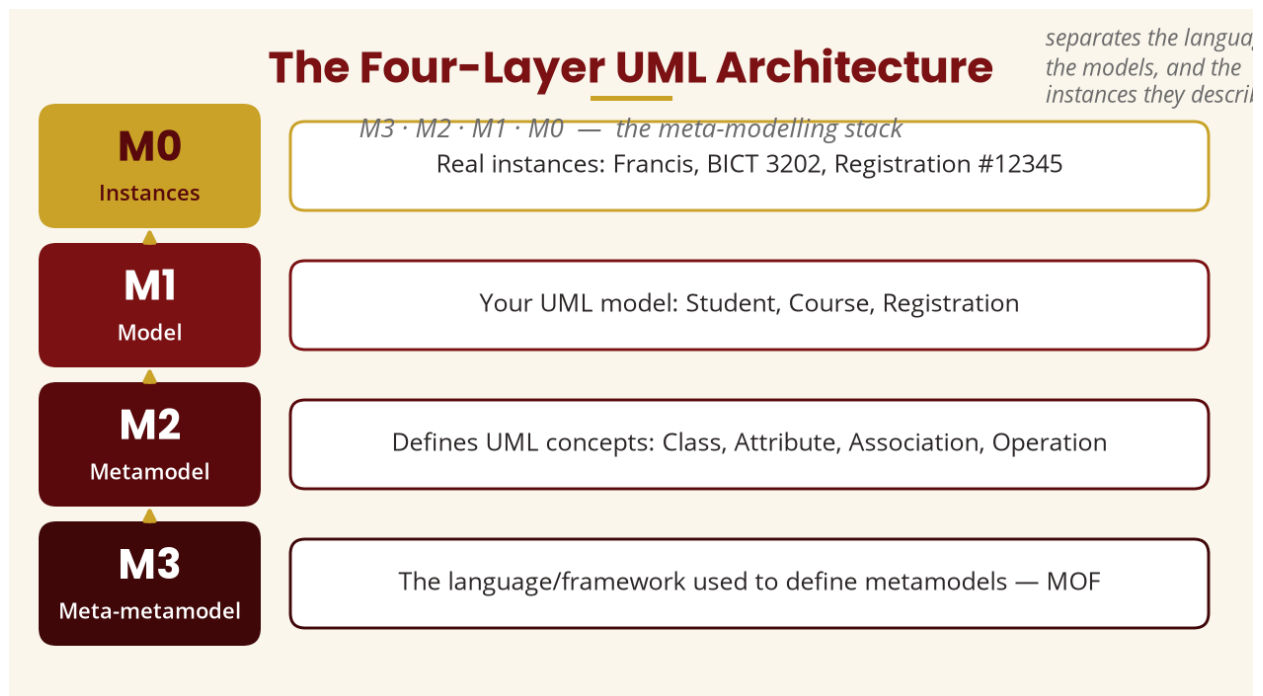


Figure 17 — The four-layer meta-modelling stack

M0 — instances: actual things — Francis : Student, Grace : Student, BICT 3202 : Course. **M1 — model:** your UML model (Student, Course, Registration). **M2 — metamodel:** the UML concepts themselves (Class, Property, Operation, Association, Generalization, UseCase, Actor, State, Activity). **M3 — meta-metamodel:** the language/framework used to define metamodels — the Meta Object Facility (MOF).

WHY IT MATTERS

The layers explain why UML is a formal language, why tools can validate models, why UML has consistent semantics, and how modelling languages themselves are defined. For introductory modelling you can draw class diagrams without mastering this — but understanding it helps in advanced model-driven engineering. **Do not confuse UML architecture with software architecture** — a common examination trap.

13. Worked Example — University Registration

For the requirement “Students should be able to register for courses online,” the same system can be shown from five UML perspectives: a **use-case** diagram (Student — Register Course), a **class** model (Student, Course with attributes and the registers relationship), a **sequence** diagram (Student □ System □ Course), an **activity** diagram (Login □ Select Course □ Check Prerequisite □ Eligible? □ Register □ Confirm), and a **state** diagram (Draft □ Submitted □ Approved □ Completed). Each answers a different question.

13.1 Practical Session — Introduction to UML Modelling

Using a UML CASE tool, a web-based UML tool or diagramming software with UML support, create a class diagram containing Student, Course, Lecturer and Registration. Then add attributes (Student: studentID, name, programme; Course: courseCode, courseName, credits; Lecturer: lecturerID, name, department; Registration: registrationID, date, status), add relationships (Student—Registration, Course—Registration, Lecturer—Course), and finally add multiplicities — reasoning about whether Student 1 — 0..* Registration and Course 1 — 0..* Registration make sense.

13.2 Tutorial — “Which Diagram Should I Use?”

Scenario	Expected diagram
What functions can a student perform?	Use Case
What classes and relationships exist?	Class
The order in which student, system and database communicate?	Sequence
The steps involved in registering a course?	Activity
The different states of an application?	State Machine
Where application components are deployed?	Deployment

14. Independent Learning (10 Hours)

- **Read** — the OMG UML 2.5.1 specification (the authoritative formal reference; it is not a beginner textbook — read it non-sequentially as the introduction advises).
- **Tabulate** — create a table of at least 10 UML diagram types: what it represents, the question it answers, when it is useful, and a university example.
- **Map** — take the Week 1–2 University Registration System and identify which diagram you would use for requirements, structure, workflow, interaction, lifecycle and deployment.

15. Week 3 Summary

Concept	Meaning
UML	Unified Modeling Language — a standardised modelling language for visualising, specifying and documenting systems.
UML is not	a programming language, a database, a development methodology, or a replacement for requirements analysis.
Building blocks	Things · Relationships · Diagrams.
Things	Structural, behavioural, grouping (package) and annotational (note).
Relationships	Association, dependency, generalization, realization.
Diagrams	Different views of the same system.
Rules	Syntax, semantics and well-formedness.
Common mechanisms	Specifications, adornments, common divisions, extensibility.
Extensibility	Stereotypes, tagged values, constraints.
Architecture	M3 meta-metamodel □ M2 metamodel □ M1 model □ M0 instances.

THE MOST IMPORTANT LESSON

UML is not about drawing pretty diagrams — it is a standardised language for communicating models of a system. Different diagrams answer different questions: what exists, how things relate, what users can do, what happens, how objects communicate, how an object changes state, what components exist, and where software runs. This prepares us for Week 4, which goes deeper into UML building blocks, rules, mechanisms and architecture before formal object/class modelling in Week 5.

16. Examination–Style Questions

Question 1. Define UML and explain its importance in OOAD (mention: Unified Modeling Language, modelling language, standardisation, visualisation, specification, documentation, communication, structure and behaviour).

Question 2. Differentiate between a modelling language and a software development methodology, using UML and UP as examples.

Question 3. Explain the three conceptual building blocks of UML.

Question 4. Explain structural, behavioural, grouping and annotational elements, with examples.

Question 5. Differentiate between syntax and semantics in UML.

Question 6. Explain the four common mechanisms of UML.

Question 7. Explain the UML four-layer architecture using a practical example.

Challenge. A university wants a mobile registration application (log in, view courses, register, drop, view status) that talks to a server and database. For each requirement, choose the most appropriate UML diagram and justify your choice.

17. References and Further Reading

Authoritative / current sources

- Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. OMG.
- Object Management Group. (n.d.). *Unified Modeling Language (UML); What is UML?; Introduction to OMG Specifications*. OMG.
- Object Management Group. (2017). *Meta Object Facility (MOF), Version 2.5.1*. OMG.

Classic OOAD reference

- Booch, G., Rumbaugh, J., & Jacobson, I. (1999). *The Unified Modeling Language User Guide*. Addison-Wesley.

Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling*.
- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML*.
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide*.
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design*.
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements*.
- Wixom, B., & Dennis, A. (2020). *Systems Analysis and Design: An Object-Oriented Approach with UML*.